Lab 6

Part I

EdgeImpulse Link: https://studio.edgeimpulse.com/public/996644/live

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

   The project uses raw accelerometer data collected from the Arduino Nano 33 BLE Sense. The raw inputs are the three accelerometer axes: accX, accY, and accZ. The data was sampled at 100 Hz over a 2.2-second interval. The dataset includes samples representing both normal fan operation and the fan being turned off.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

   The neural network (NN) classifier uses spectral features extracted from the accelerometer vibration signals through Edge Impulse's Spectral Features DSP block. The classifier receives 33 input features generated from the three accelerometer axes (accX, accY, and accZ). 5 of these features include accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, and accY RMS.

3. NN classifier outputs: State the outputs of the NN classifier.

   The outputs of the neural network classifier are the prediction classes nominal and off. The nominal output represents normal fan operation, while the off output indicates that the fan is not running. During inference, the classifier produces probability scores for each class, allowing the system to determine the most likely operating condition of the fan.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

   The anomaly detector also uses spectral features generated from the accelerometer vibration data. It analyzes selected frequency-domain features identified as important through Edge Impulse's feature importance analysis. The 4 features that were selected include accX RMS, accX Peak 1 Freq, accX Peak 1 Height, and accY Peak 3 Height.

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

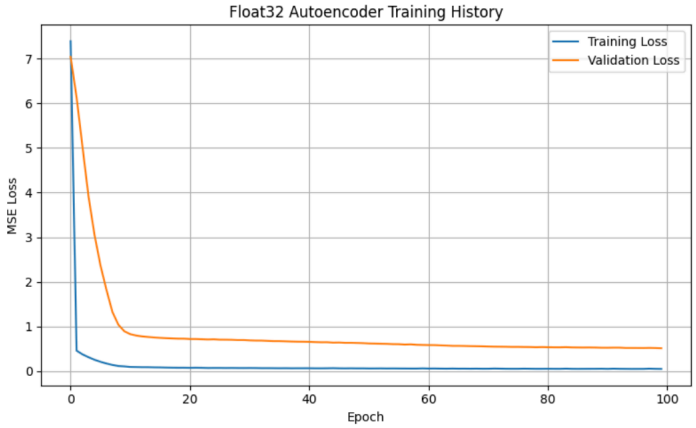
```
Anomaly prediction: 16.015026
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 583 us, anomaly 525 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 15.958893
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 593 us, anomaly 557 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 15.955334
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 564 us, anomaly 555 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.013556
Starting inferencing in 2 seconds...
```

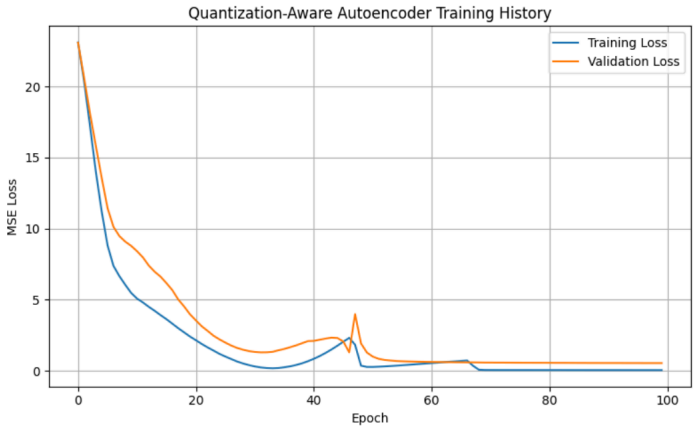
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

   The deployment results demonstrate that the machine learning model can successfully distinguish between normal fan operation and when the fan is turned off using accelerometer vibration data. However, classifier outputs should not always be completely trusted because machine learning models are only reliable when operating conditions are similar to the training environment. Changes in sensor placement, external vibrations, hardware wear, or new operating conditions can reduce prediction accuracy. The classifier is generally reliable when the device setup and fan behavior remain consistent with the training data, but it may become less accurate when exposed to unfamiliar vibration patterns or noisy environments. The anomaly detector improves system reliability by identifying unusual behavior that was not included in the original training dataset.

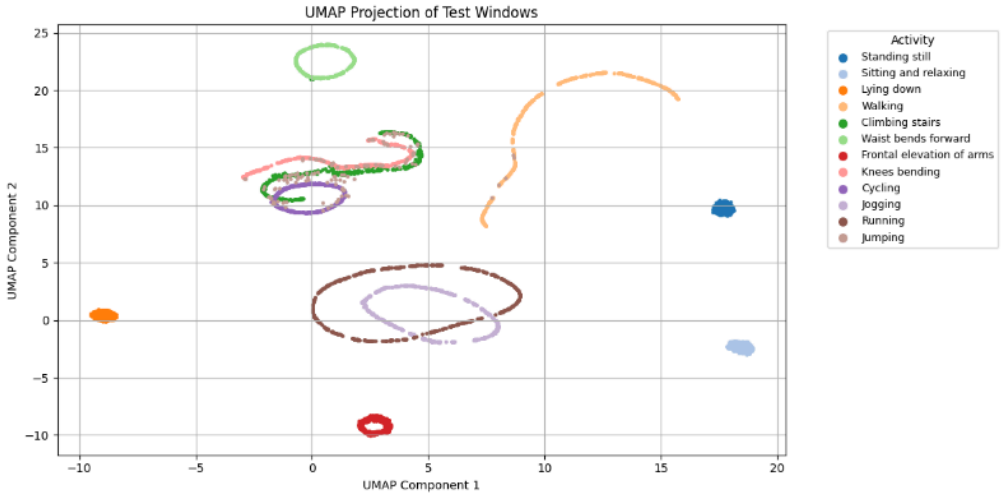
Part II

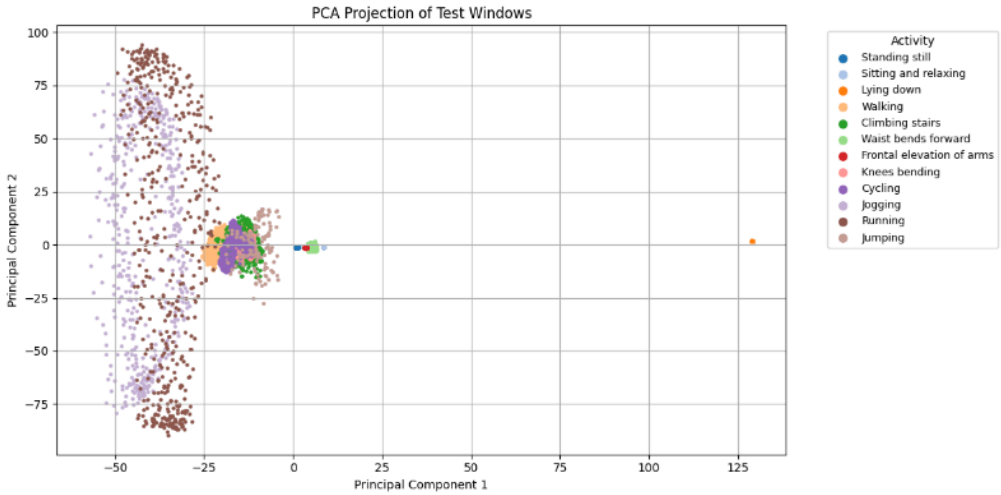
1. Training and validation loss curves.

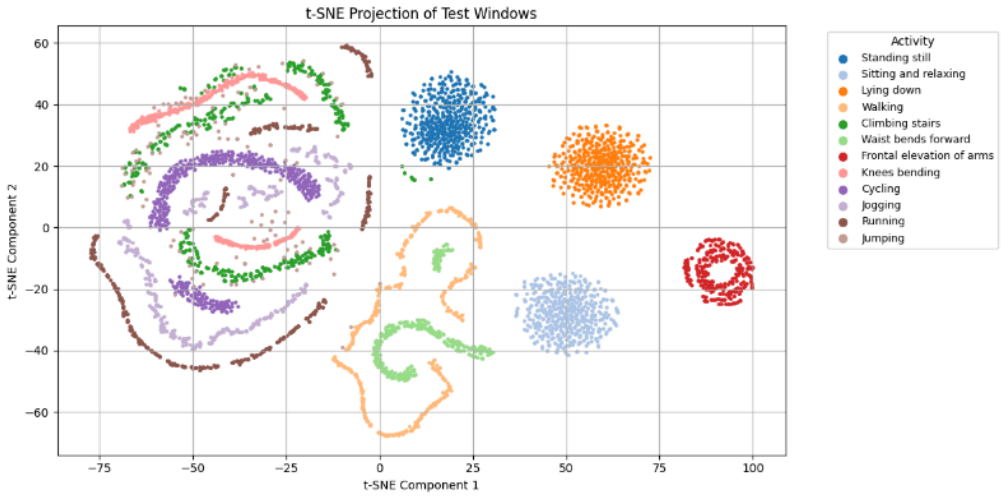




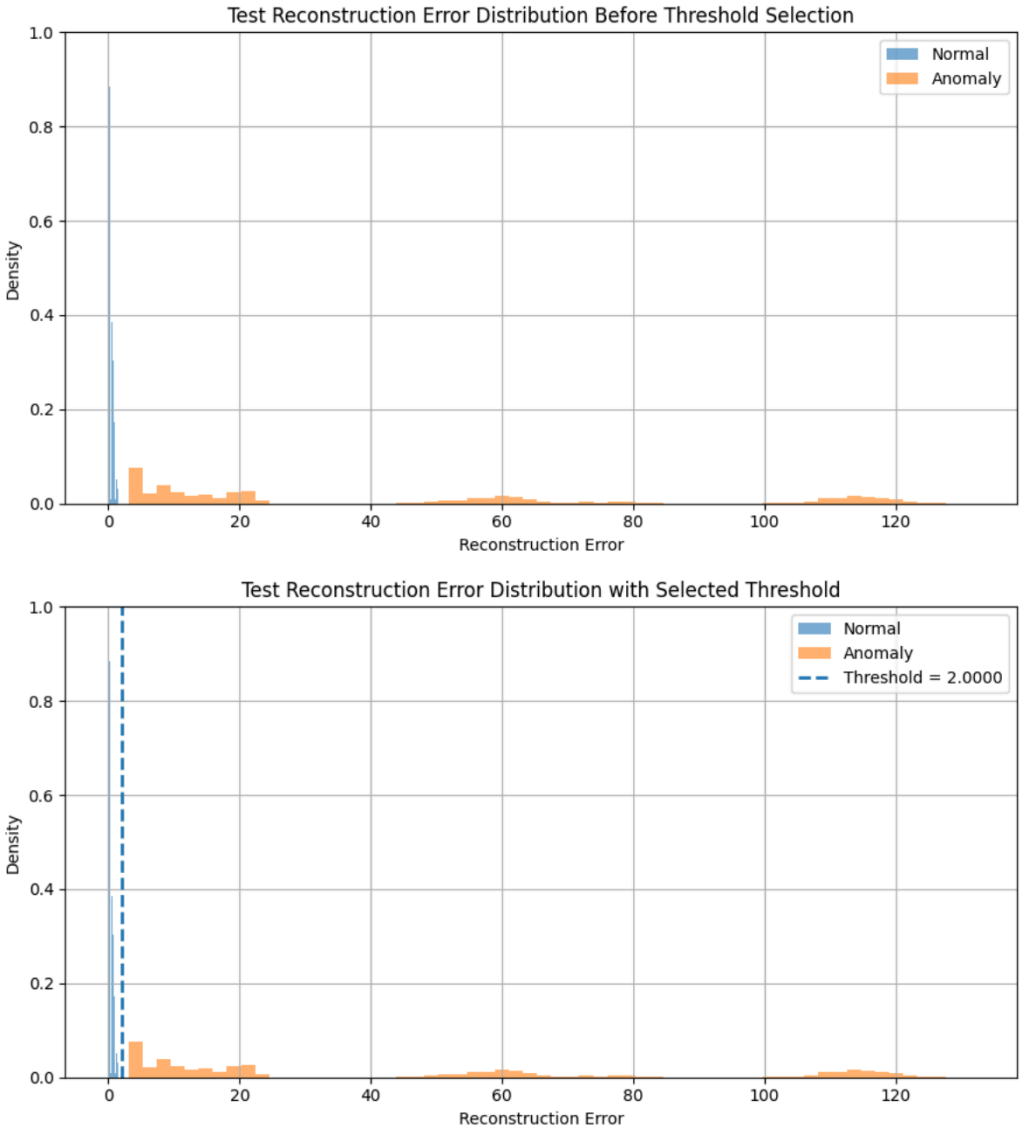
2. PCA, t-SNE, and UMAP visualizations.





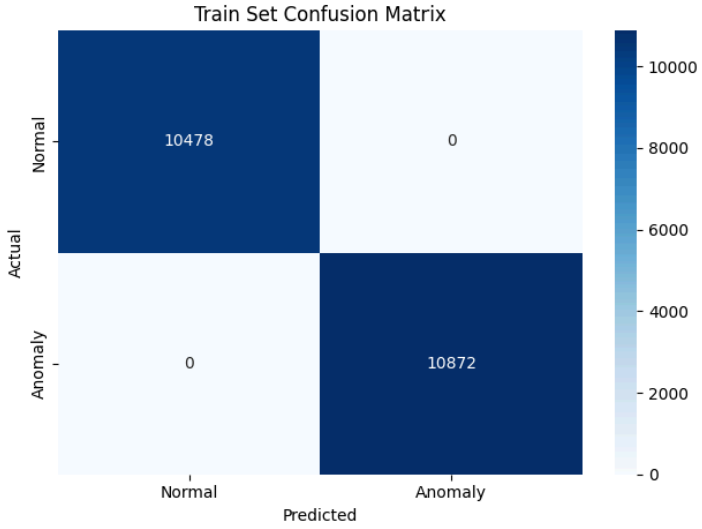


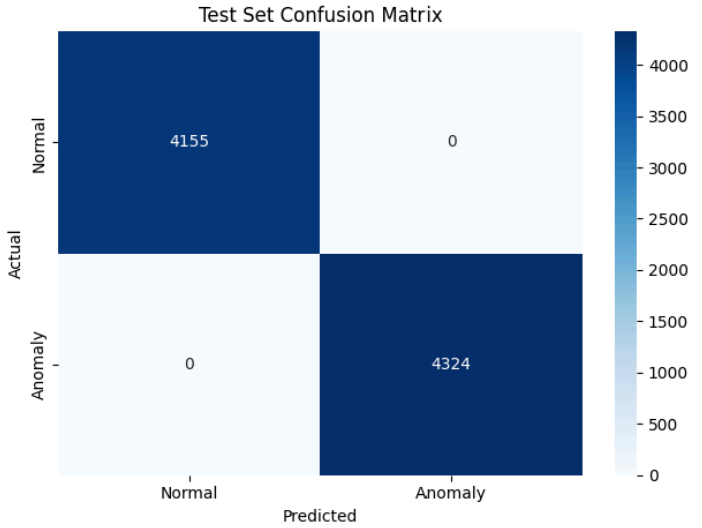
3. Reconstruction error distribution and selected threshold.



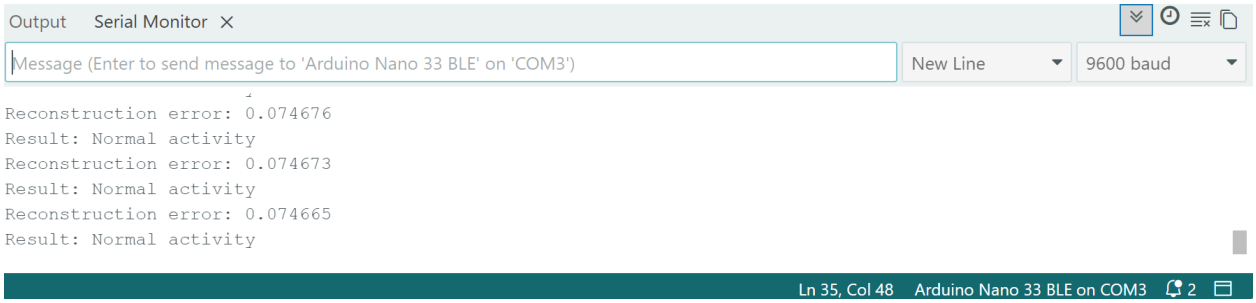
4. Confusion matrix and classification report.

```
Train set evaluation
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00     10478
 Anomaly (1)       1.00      1.00      1.00     10872

    accuracy                           1.00     21350
   macro avg       1.00      1.00      1.00     21350
weighted avg       1.00      1.00      1.00     21350

Test set evaluation
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00      4155
 Anomaly (1)       1.00      1.00      1.00      4324

    accuracy                           1.00      8479
   macro avg       1.00      1.00      1.00      8479
weighted avg       1.00      1.00      1.00      8479
```





5. Arduino serial monitor output showing reconstruction error and normal/anomaly detection result.



6. Short answers to the discussion questions listed in the submission guidelines.
   a) What threshold did you choose for anomaly detection, and why?

      I chose a reconstruction-error threshold of approximately 2.0. This value was selected after inspecting the reconstruction-error distributions for normal and anomalous samples. The normal samples had reconstruction errors mostly below about 1.5, while anomalous samples began around 3.0 and extended much higher. Since there was a clear gap between the two distributions, a threshold of 2.0 provided good separation between normal and anomalous activity while minimizing both false positives and false negatives.

   b) How does reconstruction error help distinguish normal and anomalous activity?

      The autoencoder is trained only on normal activity, so it learns how to reconstruct normal sensor patterns accurately. When normal data is passed through the model, the reconstructed output is very similar to the input, producing a low reconstruction error. However, anomalous activity differs from the patterns seen during training, so the autoencoder reconstructs it poorly, resulting in a much larger reconstruction error. By comparing the reconstruction error against a threshold, the system can classify whether the input is normal or anomalous.

   c) What information from the Python notebook must be preserved when deploying the model to Arduino?

      Components from the notebook that must be preserved during deployment include:

      - The trained TensorFlow Lite model
      - The reconstruction-error threshold value
      - Input feature ordering and dimensions
      - Window size and sampling assumptions
      - Any normalization or preprocessing steps
      - Data types and scaling factors used during quantization

      These must remain consistent so the Arduino receives data in the same format the model was trained on.

d) Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The model was trained using data processed in a very specific way. If the Arduino uses different preprocessing steps, such as different normalization values, window sizes, or feature ordering, the incoming data distribution will no longer match the training data distribution. This can cause the model to produce incorrect reconstruction errors and lead to false anomaly detections or missed anomalies. Consistent preprocessing ensures the deployed system behaves similarly to the notebook evaluation environment.

e) What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The main limitations include limited memory and computational resources on the microcontroller, reduced model accuracy due to quantization and compression, sensitivity to sensor noise and environmental changes, and potential latency constraints for real-time inference. Additionally, if the training dataset does not capture enough variation in normal behavior, the model may incorrectly classify valid activity as anomalous. TinyML devices also have strict power and storage limitations, which restrict model complexity.